

Fonksiyonlar

Genel Bakış...

2

- Giriş
- Fonksiyonlara Genel Bakış
- C ve Fonksiyonlar
- Fonksiyon Tanımı

Giriş

- Gerçek hayattaki yazılım problemlerini çözen çoğu bilgisayar programları ve yazılımları, şu ana kadar öğrendiklerimizden çok daha geniş ve karmaşık bir yapıya sahiptir.
- Tecrübeler bu tür geniş programları yazmanın en iyi yolunun, küçük parçaları ya da her biri orijinal programdan daha kolay kullanılabilecek modülleri (daha önceden hazırlanmış program parçacıkları) birleştirmek olduğunu göstermiştir.
- Bu tekniğe, **böl ve ele geçir** (**divide & conquer**) denir.

Fonksiyonlara Genel Bakış

4

- Fonksiyonlar karmaşık yapılı programların **karmaşıklığını azaltmak** ve bu **programları modüler bir yapıya** kavuşturmak için kullanılırlar.
- Fonksiyonlar, programcıların **tekrarlanan kodlar** yazmalarını önlerler.
- Fonksiyon **belirli bir adı olan** program parçasıdır.
- Fonksiyonların çalışabilmesi için bir başka fonksiyondan adı ile çağırılması gerekmektedir.

C ve Fonksiyonlar

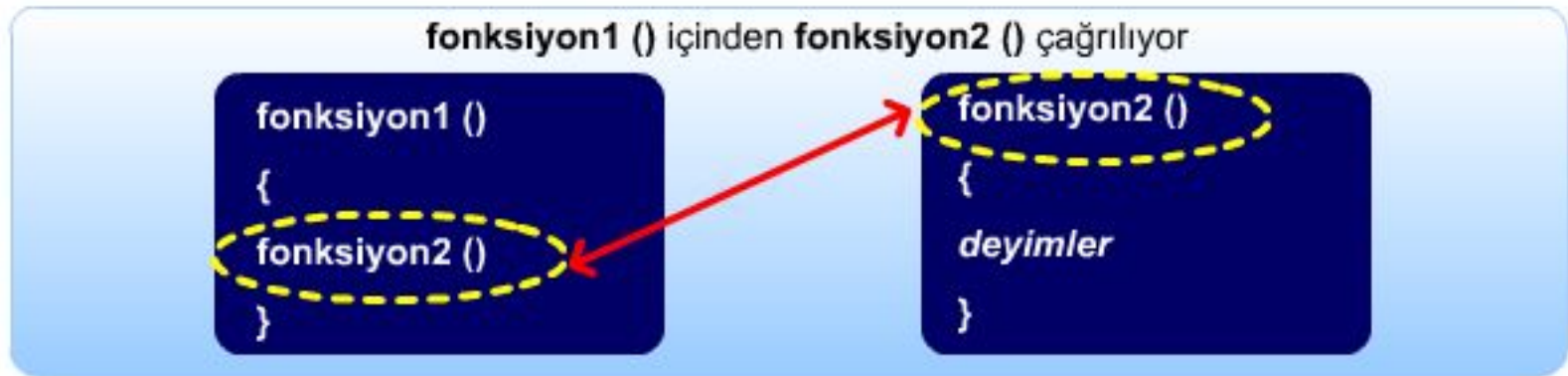
5

- C programları fonksiyonlardan oluşurlar.
- Şu ana dek kullandığımız **main()** de bir fonksiyondur. Bu fonksiyonun bir başka fonksiyon içinden çağrılmasına gerek yoktur.
- Her C programında bir **main()** fonksiyonun yer alması gerekmektedir.
- **main()** fonksiyonu, program çalıştırıldığında otomatik olarak çağrılan bir fonksiyondur.
- Bir **main()** fonksiyonu içinden bir başka fonksiyon çağrılabilir.

C ve Fonksiyonlar (devam...)

6

- Bir fonksiyon içinden bir başka fonksiyon çağrılabilir.
- Örneğin, **fonksiyon1()** isimli fonksiyondan **fonksiyon2()** isimli bir başka fonksiyon çağrılabilir.



Fonksiyon Tanımı

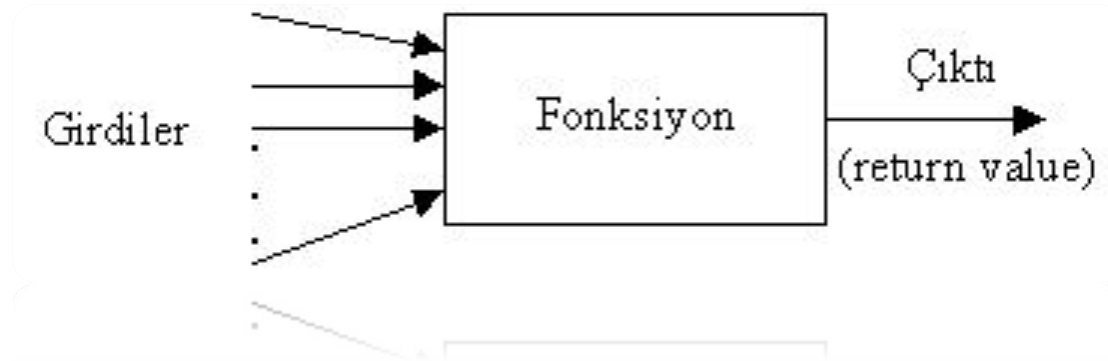
7

- Fonksiyon, belirli sayıda verileri kullanarak bunları işleyen ve bir sonuç üreten komut grubudur.
- Her fonksiyonun bir **adı** ve fonksiyona gelen değerleri gösteren **parametreleri** (bağımsız değişkenleri) vardır.
- Bir fonksiyon bu parametreleri alıp çeşitli işlemlere tabi tutar ve bir değer hesaplar.

Fonksiyon Tanımı (devam...)

8

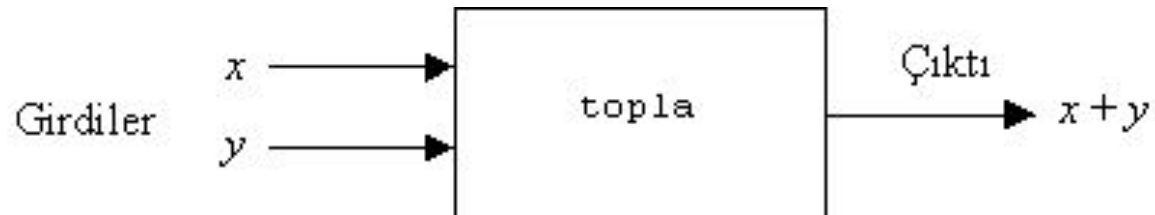
- Bu değer, çıktı veya geri dönüş değeri (**return** value) olarak adlandırılır.
- Bir fonksiyonun kaç girişi olursa olsun **sadece bir çıkışı** vardır.



Örnek-1: İki Sayının Toplamı

9

- Fonksiyon tipi: **int**
- Fonksiyon adı: **topla**
- Parametreler: **x ve y**
- Geri dönüş değeri: **x+y**



Örnek-1: İki Sayının Toplamı (devam...)

10

```
#include <stdio.h>
#include <stdlib.h>

int main()
{
    int toplam_sonucu;
    toplam_sonucu = toplama(10, 15);
    printf("Toplam sonucu:%d", toplam_sonucu);
    return 0;
}

int toplama (int x, int y)
{
    int sonuc;
    sonuc = x + y;
    return sonuc;
}
```

Parametre ve Argüman

11

- Fonksiyon çağrılırken gönderilen değerlere **Argüman** denir.
- Fonksiyon bildiriminde, fonksiyona girdi olarak, kullanılan değişkenlere **Parametre** denir.

```
#include <stdio.h>
#include <stdlib.h>

int main()
{
    int toplam_sonucu;
    toplam_sonucu = toplama(10, 15);
    printf("Toplam sonucu:%d", toplam_sonucu);
    return 0;
}

int toplama (int x, int y)
{
    int sonuc;
    sonuc = x + y;
    return sonuc;
}
```

Argüman

Parametre

Fonksiyon Bildirim Örnekleri

12

Örnek	Açıklama
<code>int islem();</code>	Tam sayı değer dönen ve parametre içermeyen bir fonksiyon
<code>int islem(void);</code>	Tam sayı değer dönen ve parametre içermeyen bir fonksiyon
<code>int islem(int x);</code>	Tam sayı değer dönen ve tam sayı türünde parametre girdisi olan bir fonksiyon
<code>void islem();</code>	Değer dönmeyen ve parametre girdisi olmayan bir fonksiyon
<code>void islem(int x);</code>	Değer dönmeyen ve tam sayı türünde parametre girdisi olan bir fonksiyon

Örnek-2: Fonksiyon Çağırımı ve Kod Akışı

13

```
#include <stdio.h>
#include <stdlib.h>

int main()
{
    mesaj();
    printf(" ye Pogramlama \n");
    return 0;
}

void mesaj()
{
    printf("Algoritma");
}
```

Algoritma ve Pogramlama

Hadi **debug** edelim...

Örnek-3: İç içe birden fazla fonksiyon çağırımı

14

```
#include <stdio.h>
#include <stdlib.h>

int main()
{
    mesaj();
    printf(" Hos geldiniz...\n");
    return 0;
}

void mesaj()
{
    printf("Algoritma");
    mesaj2();
}

void mesaj2()
{
    printf(" ve Programlama");
}
```

Algoritma ve Programlama Hos geldiniz...

Fonksiyon Geri Dönüş Değerleri

15

- Fonksiyon geri dönüş değeri **return** anahtar sözcüğü kullanılarak gerçekleştirilir.
- **return** anahtar sözcüğünün iki önemli işlevi vardır:
 1. fonksiyonun geri dönüş değerini oluşturur.
 2. fonksiyonu sonlandırır.
- **return** deyiminden sonra bir değişken, işlem, sabit veya başka bir fonksiyon yazılabilir.

Fonksiyon Geri Dönüş Değerleri (devam..)

16

- Örnekler:
 - **return** (a+b/c); /* parantez kullanmak zorunlu değil */
 - **return** 10; /* değişken kullanmak mecbur değil */
 - **return** topla(a, b)/2.0; /* önce topla fonksiyonu çalışır */
- Programın çözüm mantığına göre bir fonksiyon içerisinde birden çok geri dönüş değeri kullanılabilir.
- Fakat, ilk karşılaşılan return deyiminden sonra fonksiyon sonlanır ve çağrılan yere bu değer gönderilir.

Fonksiyon Prototip Kullanımı

17

- Bir fonksiyon prototipi, derleyiciye fonksiyon tarafından döndürülen verinin tipini, fonksiyonun almayı beklediği parametre sayısını, parametrelerin tiplerini ve parametrelerin sırasını bildirir.
- Derleyici, fonksiyonların ilk hallerini (prototiplerini) fonksiyon çağrılarını onaylamakta kullanır.
- Fonksiyon prototipinin unutulması, fonksiyonun geri dönüş tipinin int olmadığı durumda ve fonksiyon tanımı fonksiyon çağrısından daha sonra bulunmuyorsa, yazım hatalarına (syntax error) sebep olur.

Örnek-4: Fonksiyon geri değer dönüşü ve prototip kullanımı

18

```
char Notu_Harfe_Donustur (int ogrenci_notu);
```



prototip

```
int main()
{
    printf("Not: 75, Harf: %c \n", Notu_Harfe_Donustur(75));
    printf("Not: 56, Harf: %c \n", Notu_Harfe_Donustur(56));
    return 0;
}
```

```
char Notu_Harfe_Donustur (int ogrenci_notu)
{
    if( ogrenci_notu>=0  && ogrenci_notu<50 ) return 'F';
    if( ogrenci_notu>=50 && ogrenci_notu<70 ) return 'D';
    if( ogrenci_notu>=70 && ogrenci_notu<80 ) return 'C';
    if( ogrenci_notu>=80 && ogrenci_notu<90 ) return 'B';
    if( ogrenci_notu>=90 ) return 'A';
}
```

```
Not: 75, Harf: C
Not: 56, Harf: D
```

void Fonksiyonlar

19

- Bir fonksiyonun her zaman geri dönüş değerinin olması gerekmez.
- Bu durumda **return** deyimi kullanılmayabilir. Eğer bu anahtar kelime yoksa, fonksiyon ana bloğu bitince kendiliğinden sonlanır.
- Böyle fonksiyonların tipi **void** (boş, hükümsüz) olarak belirtilmelidir.
- Bu tip fonksiyonlar başka bir yerde kullanılırken, herhangi bir değişkene atanması söz konusu değildir, çünkü geri dönüş değeri yoktur. Ancak, void fonksiyonlara parametre aktarımı yapmak mümkündür.

```
#include <stdio.h>
#include <stdlib.h>
```

20

```
int main()
```

```
{
```

```
    kutu_ciz(10, 30);
```

```
    return 0;
```

```
}
```

```
void kutu_ciz(int satir, int sutun)
```

```
{
```

```
    int j;
```

```
    for ( ; satir>0; satir--){
```

```
        for (j=sutun; j>0; j--)
```

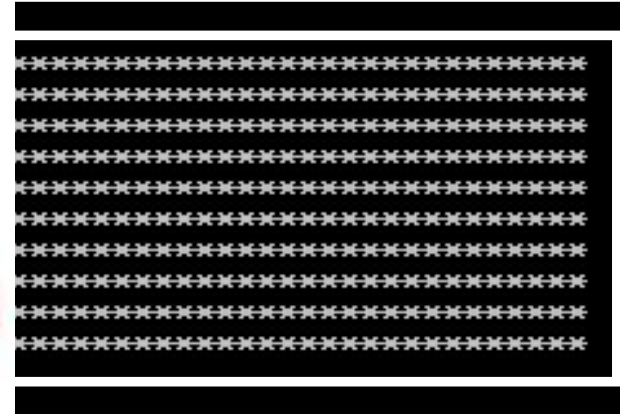
```
            printf("*");
```

```
        printf("\n");
```

```
    }
```

```
}
```

i



Fonksiyon Kullanım Hataları



21

1. Fonksiyon tanımlamalarında geri dönüş değerini unutmak.
2. Geri dönüş tipi void olarak bildirilmiş bir fonksiyonun bir değer geri döndürmesi bir yazım hatasıdır.
3. Aynı tipte fonksiyon parametrelerini *double x*, *double y* yerine *double x*, *y* olarak bildirmek. *double x*, *y* biçiminde parametre bildirmek, *y parametresinin tipinin int olmasına* sebep olur. Çünkü belirtilmeyen parametre tipi otomatik olarak int tipinde varsayılır.

Fonksiyon Kullanım Hataları (devam...)



22

5. Parametre listesini yazdığımız parantezlerin dışına noktalı virgül koymak yazım hatasıdır.
6. Bir fonksiyon parametresini daha sonradan fonksiyon içinde yerel bir değişken olarak kullanmak bir yazım hatasıdır.
7. Bir fonksiyon içinde başka bir fonksiyon tanımlamak yazım hatasıdır.
8. Fonksiyon prototipinin sonuna noktalı virgül koymamak bir yazım hatasıdır.

Fonksiyonların Yinelemesi – Kendi Kendilerini Çağırması (Recursive Function)

23

- Bazı problem tipleri için fonksiyonların kendi kendilerini çağırması kullanışlı olabilir.
- Bir yineleme fonksiyonu (recursive function), kendi kendini doğrudan ya da bir başka fonksiyon içinden çağıran fonksiyondur.
- Yineleme fonksiyonu, **bir problemi çözmek için çağrılır**.
- Bu fonksiyon, yalnızca en basit durumu ya da temel durum olarak adlandırılan durumu nasıl çözeceğini bilmektedir.
- Eğer fonksiyon temel bir durumla çağrılırsa, fonksiyon bir sonuç geri döndürür.

Fonksiyonların Yinelemesi - Kendi Kendilerini Çağırması (Recursive Function) devam...

24

- Yinelemeyi mümkün kılmak için sonraki parça orijinal probleme benzemelidir, fakat orijinal problemin daha basit ya da daha küçük bir versiyonu olmalıdır.
- Yineleme için verilebilecek en güzel örnek popüler bir matematik konusu olan Negatif olmayan bir n tamsayısının faktöriyelinin hesaplanmasıdır.
- **Örnek:**

$$5! = 5 * 4 * 3 * 2 * 1$$

$$5! = 5 * (4 * 3 * 2 * 1)$$

$$5! = 5 * (4!)$$

Örnek-6: Yinelemeli Fonksiyon Kullanarak Yapılan Faktöriyel Hesabı

25

```
#include <stdio.h>
#include <stdlib.h>
```

```
int faktoriyel_hesapla(int sayi);
```

```
int main()
```

```
{
```

```
    int sayi;
```

```
    printf("Faktoriyel hesabi icin sayi giriniz: ");
```

```
    scanf("%d", &sayi);
```

```
    printf("\nFaktoriyel sonucu: %d \n", faktoriyel_hesapla(sayi));
```

```
    return 0;
```

```
}
```

```
int faktoriyel_hesapla(int sayi)
```

```
{
```

```
    if (sayi <= 1)
```

```
        return 1;
```

```
    else
```

```
        return (sayi * faktoriyel_hesapla(sayi-1)); //yineleme kismi
```

```
}
```

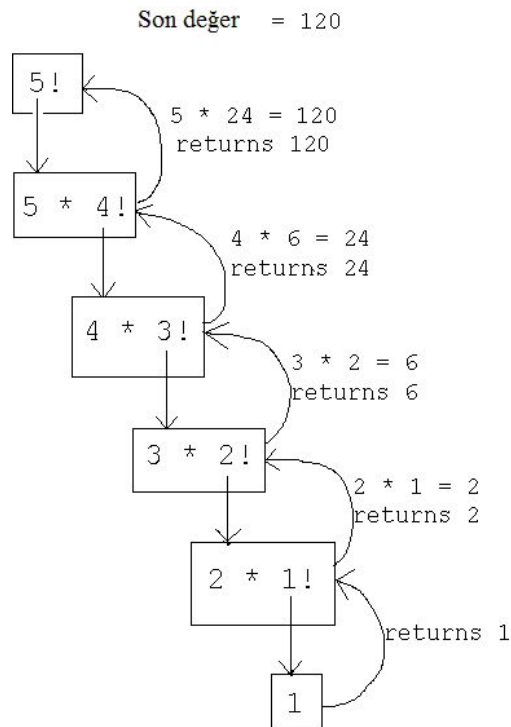
```
Faktoriyel hesabi icin sayi giriniz: 6
```

```
Faktoriyel sonucu: 720
```

```
Process returned 0 (0x0)   execution time : 4.448 s
Press any key to continue.
```

Örnek-6: Yinelemeli Fonksiyon Kullanarak Yapılan Faktöriyel Hesabı (devam...)

26



- <https://www.geeksforgeeks.org/c-recursion/>

```
// C Program to find the fibonacci number using tree
```

```
// recursion
```

```
#include <stdio.h>
```

```
int fibonacci(int n)
```

```
{
```

```
    if (n <= 1) {
```

```
        return n;
```

```
    }
```

```
    else {
```

```
        return fibonacci(n - 1) + fibonacci(n - 2);
```

```
    } }
```

```
int main()
```

```
{ int n = fibonacci(6);
```

```
    printf("%d", n);
```

```
    return 0;
```

```
}
```

```
#include<stdio.h>

int fibonacci(int);

void main ()
{
    int n,f;
    printf("Enter the value of n?");
    scanf("%d",&n);
    f = fibonacci(n);
    printf("%d",f);
}

int fibonacci (int n)
{
    if (n==0)
    {
        return 0;
    }
    else if (n == 1)
    {
        return 1;
    }
    else
    {
        return fibonacci(n-1)+fibonacci(n-2);
    }
}
```

```
#include<stdio.h>
```

```
void fun1(int n)
{
    if(n>0)
    {
        printf("%d ",n);
        fun1(n-1);
    }
}

main()
{
    int x=5;
    fun1(x);
}
```

```
#include <stdio.h>
int sum(int n);

int main()
{
    int number, result;

    printf("pozitif bir tamsayı girin: ");
    scanf("%d", &number);

    result = sum(number);

    printf("Toplam = %d", result);
    return 0;
}

int sum(int num)
{
    if (num!=0)
        return num + sum(num-1); // sum() function calls itself
    else
        return num;
}
```

Yineleme mi Tekrar mı?

31

- Tekrar ve yinelemenin ikisi de döngü içerir.
- Tekrar özellikle döngü yapısını kullanırken, yineleme döngüyü fonksiyon çağrılarının tekrarında kullanır.
- Tekrar ve yinelemenin ikisi de bir sonlandırma testi içerirler.
- Yineleme temel bir durumla karşılaşıldığında, tekrar ise döngü devam koşulu yanlış hale geldiğinde sona erer.
- Yineleme bir çok negatif özelliğe sahiptir.
- Yineleme, mekanizmayı sürekli çağırarak fonksiyon çağrılarının artmasına sebep olur. Bu, işlemci zamanı ve hafızada fazladan yük demektir.

Yineleme mi Tekrar mı? (devam...)

32

- Her yineleme çağrısı, fonksiyonun başka bir kopyasının oluşmasına sebep olur, bu da hafızayı fazladan işgal etmek demektir.
- Tekrar, fonksiyonların sürekli olarak çağrılması ve fazladan hafıza kullanılması engellenir.
- Yinelemeli olarak çözülen her problem tekrarlı bir biçimde çözülebilir. Yineleme yaklaşımı genelde problemi daha iyi yansıttığı ve daha kolay anlaşılan ve hataları kolay ayıklanan programlar yazılmasını sağlattığı için, tekrar yaklaşımına göre tercih edilebilir.
- Yinelemeli çözümleri seçmenin başka bir sebebi de tekrarlı çözümün kolaylıkla bulunamayışıdır.

Dizilerin Fonksiyonlarda Kullanılması

33

- Diziler de sıradan değişkenler gibi bir fonksiyona parametre olarak aktarılabilirler.
- Fakat, aktarma kuralı biraz farklıdır.

*Her zaman dizinin yanında
boyutunun da bilinmesi gereklidir !!!*

Örnek 7. Dizi'nin Fonksiyonda Kullanılması

```
#include <stdio.h>
#include <stdlib.h>

void dizi_yaz(float x[], int n);

int main() {
    float dizi[5]= { 8.471, 3.683, 9.107, 4.739, 3.918 };
    dizi_yaz(dizi, 5);
    return 0;
}

void dizi_yaz(float x[], int n)
{
    int i;
    for(i=0; i<n; i++)
        printf("%.2f \n", x[i]);
    printf("\n");
}
```

Tavsiyeler

35

- ✓ Birden fazla fonksiyon kullanılan programlarda, **main** fonksiyonu programın esas görevini yerine getiren fonksiyonların çağırıcısı olarak kullanılmalıdır.
- ✓ Her fonksiyon, iyi olarak tanımlanmış tek bir işi yapacak şekilde sınırlandırılmalıdır ve fonksiyon ismi, fonksiyonun görevini etkili bir biçimde açıklamalıdır. Bu, **özetlemeyi** ve yazılımın **yeniden kullanılabilirliğini** sağlar.
- ✓ Eğer fonksiyonun görevini açıklayacak etkili bir isim bulamıyorsanız muhtemelen yazdığınız fonksiyon birden fazla görevi yerine getirmeye çalışmaktadır. Bu tarzda fonksiyonları daha küçük fonksiyonlara bölmek en iyi yoldur.

Tavsiyeler (devam...)

36

- ✓ Bir fonksiyon genellikle bir sayfadan daha uzun olmamalıdır. Küçük fonksiyonlar yazılımın **yeniden kullanılabilmesini** sağlar.
- ✓ Programlar, küçük fonksiyonların bir araya getirilmesiyle yazılmalıdır. Bu, programların daha kolay yazılması, **değiştirilmesi** ve **hatalarının giderilmesini** sağlar.
- ✓ Çok fazla sayıda parametreye ihtiyaç duyan fonksiyonlar birden fazla görevi yerine getirebilir. Böyle fonksiyonları ayrı görevleri gerçekleştiren daha küçük fonksiyonlara bölmek gerekir. Fonksiyonun başlığı mümkünse bir satıra sığmalıdır.

```
□  
□  
□  
□  
□ #include<stdio.h>  
□ int main()  
□ {  
□     static int i=1; // *  
□     printf("%d\n",i++);  
□     if(i==6)  
□     {  
□         return 0;  
□     }  
□     main(); // recursive call of main()  
□ }  
□
```

- Klavyeden girilen bir sayının asal olup olmadığını fonksiyon kullanarak bulan programı yazdırınız.

```
#include <stdio.h>
int checkPrime(int n);
int main()
{int n, i;
printf("pozitif bir deger giriniz ");
scanf("%d", &n);
```

```
if (checkPrime(n) == 1)
printf("%d asaldir",n);
else
printf("%d asal DEGILDIR",n);
return 0;
}
int checkPrime(int n)
{
int i, isPrime = 1;
for(i = 2; i <= n/2; ++i)
{
if(n % i == 0)
{
isPrime = 0;
break;
}}
return isPrime;
}
```

- **Klavyeden girilen bir sayı iki asal sayının toplamı şeklinde yazılabilir mi?**

```

#include <stdio.h>
int checkPrime(int n);
int main()
{
    int n, i, flag = 0;
    printf("Enter-Deger giriniz.....: ");
    scanf("%d", &n);

    for(i = 2; i <= n/2; ++i)
    {
        if (checkPrime(i) == 1)
        {
            // condition for n-i to be a prime number
            if (checkPrime(n-i) == 1)
            {
                // n = primeNumber1 + primeNumber2
                printf("%d = %d + %d\n", n, i, n - i);
                flag = 1;
            }
        }
        if (flag == 0)
            printf("%d iki asal sayinin toplami olarak yazilamaz.", n);
        return 0;
    }
    // Function to check prime number
    int checkPrime(int n)
    {
        int i, isPrime = 1;

        for(i = 2; i <= n/2; ++i)
        {
            if(n % i == 0)
            {
                isPrime = 0;
                break; }
        }

        return isPrime;
    }
}

```

3. Verilen bir N sayısının karekökünü bulmak için Babil döneminden beri kullanılan tekrarlamalı (iteratif) bir yöntem bulunmaktadır. (25P)

Bu yöntemde aşağıdaki gibi tekrarlı olarak çalışmaktadır.

- I. Karekökü alınacak sayı ve bu karekök için bir tahmin değeri girilir (x_0) girilir.
- II. Bir sonraki tahmin değeri ise aşağıdaki gibi hesaplanmaktadır :

$$x_1 = \frac{1}{2} \left(x_0 + \frac{N}{x_0} \right)$$

Bu denklem genelleştirilerek; $x_{n+1} = \frac{1}{2} \left(x_n + \frac{N}{x_n} \right)$ şeklinde yazılabilir.

- III. Hesaplanan yeni tahmin değerinin karesi, sayıdan çıkartılarak hata değeri hesaplanmaktadır.
- IV. Belirlenen hata değerinden daha küçük bir değere ulaşıncaya kadar II ve III numaralı işlemler tekrarlanmaktadır.
- V. Hata değeri 0.002 değeri ile klavyeden girilen sayı ve tahmin değeri ile karekök hesabı yapan programı yazınız.

Kombinasyon nasıl hesaplanır?

Kombinasyon, n asıl kümenin eleman sayısı ve r alt kümelerin eleman sayısı olmak üzere

$$C(n, r) = \binom{n}{r} = \binom{n}{n-r} = \frac{P(n, r)}{r!} = \frac{n!}{r!(n-r)!}$$

formülüyle hesaplanır.

Permütasyon nasıl hesaplanır?

Permütasyon, n asıl kümenin eleman sayısı ve r alt kümelerin eleman sayısı olmak üzere

$$P(n, r) = \binom{n}{n-r} = \frac{n!}{(n-r)!}$$

formülü ile hesaplanır.

- Klavyeden girilen n ve r değerleri için fonksiyon permutasyon hesaplayınız

```
□ #include<stdio.h>
□ #include<conio.h>
□ main()
□ {
□     int n,c,r,npr;
□     long int fact( int ); /* function Prototype */
□     printf("n ve r degerilerini giriniz n>r olmali :\n");
□     scanf("%d%d",&n,&r);
□     npr= fact(n)/fact(n-r);

□     printf("permutasyon = %d\n",npr);
□     long int fact( int x)
□     {
□         long int f=1;
□         int i;
□         for(i=1;i<=x;i++)
□             f=f*i;
□         return(f);
□     }
```

KAYNAKLAR

46

- N. Ercil Çağıltay ve ark., C DERSİ PROGRAMLAMAYA GİRİŞ, Ada Matbaacılık, ANKARA; 2009.
- Milli Eğitim Bakanlığı "Programlamaya Giriş ve Algoritmalar Ders Notları", 2007
- <http://tr.wikipedia.org/wiki/Code::Blocks>
- <http://www.codeblocks.org>
- <http://www.AlgoritmaveProgramlama.com>
- <http://www1.gantep.edu.tr/~bingul/c>



Algoritma ve Programlama